Initial raw data clean-up

Overview

This notebook describes my tidying of the raw data from the fall 2026 CURIOSS OSPO survey. Mostly I'm just taking the data out of excel and writing it to tab-separated files for easy downstream analysis with r. I'm writing them to my data/ folder in the root of this project. (Not checked into version control.)

These data could go in a relational database, but why overcomplicate things? tsv files will do the job.

N.B.: I already did a tiny bit of manual clean-up by removing three artifactual rows: one was from me when I was just clicking around, not actually taking the survey, one was from a respondent who started the survey, didn't finish, then came back and completed the whole thing from scratch (I deleted their first attempt), and the first row, which only had real data for the ranking question. I suspect this was a survey administrator making sure that that question worked.

Import packages, utilities, and config variables

```
library(readxl)
library(dplyr)
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':

    filter, lag
```

```
The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(docstring)
```

```
Attaching package: 'docstring'

The following object is masked from 'package:utils':

    ?
```

```
library(stringr)

source("utils.R")

# Read config.yml
cfg <- config::get()
```

Prepare raw data

After receiving the data from Clare, a password-protected .xlsx file, I copied the data into a new excel file without password protection so R's readxl package could read it. That file is in data/ and is called raw_data.xlsx.

Right off the bat, I manually removed three non-empty but spurious rows: - Me! I didn't take the survey but I was clicking around in it. - Somebody who took the survey twice, I see - The very first response, which I suspect was a test by one of the survey organizers. The organization is listed as "dsad" and the email is "dsas", and the locations of those keys on a qwerty keyboard support my hypothesis that that was just some nonsense for a test run. The only question that that person answered was the "choose your top 3" drag-and-drop, so I assume an organizer was just testing that that question worked.

Import codebook

The column headers in the raw data are too long to be tractable in a programmatic environment. They need short nicknames without special characters or spaces. Rather than hard-coding those nicknames into this script, I am putting them in another excel file. That file must

be called `codebook.xlsx`. It has a single sheet with six columns: `nickname`, `root`, `tip`, `output_file`, and `category`.

`nickname` is a short, unique, machine-friendly name for the column (no spaces or special characters).

`root` is the question text as it appears in the survey. For questions with sub-items (e.g. select-all-that-apply or matrix questions), `root` is just the question stem — the part before the bracketed sub-item.

`tip` is the sub-item text from inside the brackets, for questions that have them. Leave it empty for questions without sub-items. If the tip is exactly `Other`, that column is automatically routed to its `output_file` without any extra configuration.

`output_file` is the basename of the TSV file this column should be written to (without the `.tsv` extension). For most questions this is the question group name (e.g. `comm_building`, `objectives`). Write-in "other" columns are collected into combined files: all SATA `[Other]` columns share `output_file = sata_other`, all `mc_complex [Other]` columns share `mc_other`, and matrix `[Other]` columns share `matrix_other`. Qualitative follow-up questions that don't belong to a question group can use `output_file = qual`.

`category` describes the question type and must be one of these seven values:
`mc_simple`: simple multiple choice with no "other" write-in option.
`mc_complex`: multiple choice with an "other" write-in option.
`qual_pii`: qualitative responses with personally identifiable information of high sensitivity.
`qual`: other qualitative responses.
`matrix`: matrix questions.
`ranking`: drag-and-drop ranking questions (e.g. choose your top three).
`sata`: select-all-that-apply questions.

```
codebook <- read_excel(file.path(cfg$data_path, "codebook.xlsx"))
```

Define functions

```
clean_p_tags <- function(df) {
  #' Clean HTML P Tags from Data Frame Entries
  #'
  #' Removes opening `<p>` (including any HTML attributes) and closing `</p>` tags
  #' from the start and end of all character or factor columns in a data frame.
  #'
  #' @param df A data frame containing character or factor columns to clean.
  #'
```

```r
  #' @return A data frame with outer `<p>` and `</p>` tags stripped from all string entries.
  #'
  #' @examples
  #' df <- data.frame(
  #'   title = c("<p>First Title</p>", "Second Title"),
  #'   description = c("<p class='intro'>Body text</p>", "No tags")
  #' )
  #' clean_p_tags(df)

  df[] <- lapply(df, function(col) {
    if (is.character(col) || is.factor(col)) {
      # Remove leading <p> (with optional attributes/whitespace) and trailing </p>
      col <- gsub("^\\s*<p[^>]*>", "", col, ignore.case = TRUE)
      col <- gsub("</p>\\s*$", "", col, ignore.case = TRUE)
    }
    return(col)
  })
  return(df)
}

select_and_rename <- function(df, name_map) {
  #' Select and Rename Columns Using a Named Vector
  #'
  #' @description
  #' Extracts a subset of columns from a data frame based on the values in a
  #' named vector and renames them to match the corresponding vector names.
  result <- df |>
    select(any_of(unname(name_map))) |>
    rename(any_of(name_map))

  return(result)
}

reconstruct_original_name <- function(root, tip) {
  #' Reconstruct Original Survey Column Name
  #'
  #' @description
  #' Combines a question root and an optional tip (sub-item text) into the
  #' verbatim column header used in the raw survey export. If tip is NA or
  #' empty, the root is returned as-is. Otherwise, returns "root [tip]".
  #'
  #' @param root Character string: the question stem.
```

```
  #' @param tip  Character string or NA: the bracketed sub-item text.
  #' @return A character string matching the raw survey column header.
  if (is.na(tip) || trimws(tip) == "") {
    return(root)
  }
  paste0(root, " [", tip, "]")
}

get_name_map <- function(out_file, codebook_df) {
  #' Generate a Named Vector of Column Mappings for an Output File
  #'
  #' @description
  #' Filters the codebook to rows with the given output_file, reconstructs
  #' the original survey column name from root and tip, and returns a named
  #' vector of the form c(nickname = original_name) suitable for use with
  #' select_and_rename().
  #'
  #' @param out_file   Character string matching the 'output_file' column.
  #' @param codebook_df The codebook data frame.
  #' @return A named character vector.
  subset_df <- codebook_df |>
    filter(output_file == out_file) |>
    mutate(original_name = mapply(reconstruct_original_name, root, tip))

  setNames(subset_df$original_name, subset_df$nickname)
}
```

Read data from excel

```
data <- readxl::read_excel("data/raw_data.xlsx", col_types = "text", na = c(""))
# Using a workaround to preserve distinction between
# "" and "N/A", because by default read_excel converts
# both of these to NA objects.
data[is.na(data)] <- ""
data[data == "N/A"] <- NA

# Remove Windows line endings (\r\n), Unix line endings (\n), and carriage returns (\r).
data <- data |>
  mutate(across(where(is.character), ~ str_replace_all(.x, "[\r\n]+", " ")))
```

Initial clean-up

Let's just look at the data first.

```
dim(data)
```

```
[1]  43 198
```

```
names(data)[1:15]
```

```
 [1] "Response ID"
 [2] "Date submitted"
 [3] "Last page"
 [4] "Start language"
 [5] "Seed"
 [6] "Quota exit"
 [7] "Your organization / institution name"
 [8] "Your email"
 [9] "In which region is your organization located?"
[10] "In which region is your organization located? [Other]"
[11] "Which best describes the open source function at your organization?"
[12] "Which best describes the open source function at your organization? [Other]"
[13] "Where is your OSPO located within your university? [University library]"
[14] "Where is your OSPO located within your university? [Central administration (e.g., centr
[15] "Where is your OSPO located within your university? [Technology transfer, innovation, or
```

Remove uninformative columns.

```
cols_to_remove <- c(
    "Response ID",
    "Date submitted",
    "Last page",
    "Start language",
    "Seed",
    "Quota exit")

data <- data |> select(-any_of(cols_to_remove))
dim(data)
```

```
[1]  43 192
```

Remove uninformative rows.

```
data <- remove_uninformative_rows(data)
dim(data)
```

```
[1]  33 192
```

Remove HTML

tags that are present in some of the responses (presumably a bug from limesurvey).

```
data <- clean_p_tags(data)
```

Write output files

Each row in the codebook table specifies which output file a survey column belongs to. The loop below iterates over each distinct `output_file` value, extracts the corresponding columns from the data, and writes them to a TSV file. The order of output files follows their first appearance in the mappings table.

```
output_files <- unique(codebook$output_file)

for (out_file in output_files) {
  name_map <- get_name_map(out_file, codebook)
  df <- select_and_rename(data, name_map)
  write_df_to_file(df, paste0(out_file, ".tsv"))
}
```